

টোকেনাইজার (Tokenizer) ন্যাচারাল ল্যাঙ্গুয়েজ প্রসেসিং (NLP)-এর একটি অত্যন্ত গুরুত্বপূর্ণ অংশ। নিচে এর মূল বিষয়গুলো সহজভাবে বুঝিয়ে বলা হলো:

- টোকেনাইজার কেন প্রয়োজন: আমরা যখন NLP নিয়ে কাজ করি, তখন আমাদের প্রধান ডেটা বা তথ্য থাকে সাধারণ টেক্সট বা লেখা হিসেবে। কিন্তু মেশিন লার্নিং মডেলগুলো মানুষের এই কাঁচা টেক্সট সরাসরি পড়তে বা বুঝতে পারে না; তারা শুধুমাত্র সংখ্যা (**numbers**) নিয়ে কাজ করতে পারে।
- টোকেনাইজারের মূল লক্ষ্য: টোকেনাইজারের প্রধান কাজ হলো মানুষের সাধারণ ভাষাকে সংখ্যায় রূপান্তর করা। এর উদ্দেশ্য হলো টেক্সটের এমন একটি গাণিতিক রিপ্রেজেন্টেশন বা রূপ খুঁজে বের করা, যা সবচেয়ে বেশি অর্থবহ (meaningful) হয়।
- টোকেনাইজেশনের তিনটি প্রধান পদ্ধতি: এই টেক্সট-টু-নাম্বার রূপান্তরের জন্য তিনটি ভিন্ন ধরনের অ্যালগরিদম ব্যবহার করা হয়, যা আপনাকে নিচের ক্রমানুসারে দেখার পরামর্শ দেওয়া হয়েছে:
 - Word-based (শব্দ-ভিত্তিক):** এখানে প্রতিটি শব্দকে আলাদা টোকেন হিসেবে ধরা হয়।
 - Character-based (অক্ষর-ভিত্তিক):** এখানে প্রতিটি অক্ষরকে আলাদাভাবে প্রসেস করা হয়।
 - Subword-based (উপ-শব্দ ভিত্তিক):** এটি শব্দ এবং অক্ষরের মাঝামাঝি একটি আধুনিক পদ্ধতি যেখানে বড় শব্দকে ছোট ছোট অর্থপূর্ণ অংশে ভাগ করা হয়।

সংক্ষেপে, টোকেনাইজার হলো একটি অনুবাদকের মতো যা মানুষের ভাষাকে কম্পিউটারের বোঝার উপযোগী সংখ্যায় বদলে দেয় যাতে মডেলটি সেই ভাষা নিয়ে কাজ করতে পারে।

1. Word-based (শব্দ-ভিত্তিক):

Word-based tokenization টেক্সটকে সংখ্যায় রূপান্তর করার একটি প্রাথমিক পদ্ধতি। নিচে এর কাজ করার প্রক্রিয়া এবং সীমাবদ্ধতাগুলো তুলে ধরা হলো:

Split on spaces				
Let's	do	tokenization!		
Split on punctuation				
Let	's	do	tokenization	!

১. কাজ করার প্রক্রিয়া:

- এই পদ্ধতিতে কাঁচা টেক্সটকে স্পেস (space) বা বিরামচিহ্নের ওপর ভিত্তি করে আলাদা আলাদা শব্দে ভাগ করা হয়।
- প্রতিটি শব্দের জন্য একটি নির্দিষ্ট **ID** বা সংখ্যা বরাদ্দ করা হয়। যেমন: "Let's" শব্দের জন্য একটি আইডি এবং "do" শব্দের জন্য অন্য একটি আইডি থাকে।
- এর বড় সুবিধা হলো, প্রতিটি সংখ্যার মধ্যে ওই শব্দটির পূর্ণাঙ্গ অর্থ এবং প্রাসঙ্গিক তথ্য লুকিয়ে থাকে।

২. সীমাবদ্ধতা ও সমস্যা: এই পদ্ধতির বেশ কিছু বড় সমস্যা রয়েছে:

- সম্পর্ক বুঝতে না পারা: "dog" এবং "dogs" শব্দ দুটি প্রায় একই অর্থ বহন করলেও, এই পদ্ধতিতে তারা সম্পূর্ণ আলাদা আইডি পায়। ফলে মডেল সহজে বুঝতে পারে না যে "dogs" আসলে "dog" শব্দেরই একটি বহুবচন রূপ।
- বিশাল ভোকাবুলারি (**Large Vocabulary Size**): একটি ভাষায় অসংখ্য শব্দ থাকে। যদি প্রতিটি শব্দের জন্য আলাদা আইডি রাখা হয়, তবে মডেলের শব্দভাণ্ডার বা ভোকাবুলারি অনেক বড় হয়ে যায়। এই বিশাল তালিকা সামলানোর জন্য মডেলের অনেক বেশি মেমরি এবং ওয়েট (weights) প্রয়োজন হয়।
- অজানা শব্দ (**Unknown words**): মেমরি বাঁচাতে আমরা যদি শুধুমাত্র সবচেয়ে বেশি ব্যবহৃত শব্দগুলো (যেমন: শীর্ষ ১০,০০০ শব্দ) নিয়ে ভোকাবুলারি তৈরি করি, তবে এর বাইরের শব্দগুলোকে মডেল "unknown" বা অজানা শব্দ হিসেবে ধরে নেয়। এর ফলে অনেক গুরুত্বপূর্ণ তথ্য হারিয়ে যায়, কারণ মডেল সব অজানা শব্দের জন্য একই রকম রিপ্রেজেন্টেশন ব্যবহার করে।

সংক্ষেপে, শব্দ-ভিত্তিক টোকেনাইজেশন সহজ হলেও এটি শব্দের মধ্যকার সূক্ষ্ম সম্পর্ক বুঝতে পারে না এবং অজানা শব্দের ক্ষেত্রে তথ্য হারিয়ে ফেলে। এই সমস্যাগুলো সমাধানের জন্যই পরবর্তীকালে **Character-based** -এর মতো উন্নত পদ্ধতির প্রয়োজন হয়।

2.Character-based (অক্ষর-ভিত্তিক):

Character-based tokenization (অক্ষর-ভিত্তিক টোকেনাইজেশন) শব্দ-ভিত্তিক পদ্ধতির সমস্যাগুলো কাটিয়ে উঠতে এই পদ্ধতিটি কীভাবে কাজ করে এবং এর সুবিধা-অসুবিধাগুলো নিচে বিস্তারিত দেওয়া হলো:

L	e	t	'	s	d	o	t	o	k	e	n	i	z	a	t	i	o	n	!
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

- কাজের প্রক্রিয়া: এই পদ্ধতিতে টেক্সটকে শব্দে ভাগ করার বদলে প্রতিটি আলাদা অক্ষরে (**individual characters**) ভাগ করা হয়।
- সুবিধাসমূহ:
 - ছোট ভোকাবুলারি: যেকোনো ভাষায় শব্দের সংখ্যা লাখ লাখ হলেও অক্ষরের সংখ্যা খুবই সীমিত থাকে। যেমন, প্রায় ১,৭০,০০০ ইংরেজি শব্দের বিপরীতে মাত্র ২৫৬টি অক্ষর দিয়েই পুরো ভাষা কভার করা সম্ভব। এমনকি চীনা ভাষার মতো কঠিন ভাষার ক্ষেত্রেও অক্ষরের সংখ্যা শব্দের তুলনায় অনেক কম হয়।
 - অজানা শব্দের সমস্যা দূর করা: যেহেতু ভাষার সব অক্ষর ভোকাবুলারিতে থাকে, তাই কোনো নতুন শব্দ বা ভুল বানানের শব্দ (misspelled words) সামনে এলেও মডেল সেটি টোকেনাইজ করতে পারে। এটি শব্দ-ভিত্তিক পদ্ধতির মতো কোনো শব্দকে সরাসরি "unknown" হিসেবে ফেলে দেয় না।
- সীমাবদ্ধতাসমূহ:

- তথ্যের অভাব: একটি শব্দ এককভাবে যতটা তথ্য বহন করতে পারে, একটি অক্ষর তা পারে না। যেমন: "Let's" শব্দটি যতটা অর্থবহ, শুধু "l" অক্ষরটি ততটা নয়। রোমান-ভিত্তিক ভাষার ক্ষেত্রে মডেলকে একটি শব্দের অর্থ বুঝতে অনেকগুলো টোকেন বা অক্ষর একসাথে প্রসেস করতে হয়।
- বিশাল সিকোয়েন্স লেন্থ: অক্ষর-ভিত্তিক পদ্ধতিতে একটি ছোট বাক্যও অনেক বড় টোকেন লিস্টে পরিণত হয়। এর ফলে মডেলের মেমরি বা **Context Size** দ্রুত শেষ হয়ে যায় এবং ইনপুট হিসেবে আমরা খুব বেশি বড় টেক্সট ব্যবহার করতে পারি না।

পরিশেষে বলা হয়েছে যে, কিছু সীমাবদ্ধতা থাকলেও এই পদ্ধতিটি অতীতে বেশ ভালো ফলাফল দিয়েছে এবং শব্দ-ভিত্তিক পদ্ধতির কিছু জটিল সমস্যা সমাধানে এটি বেশ কার্যকর। মূলত এই দুই পদ্ধতির (Word and Character) সমন্বয় করেই তৈরি হয়েছে পরবর্তী ধাপের **Subword-based tokenization**।

3. Subword-based (উপ-শব্দ ভিত্তিক):

Subword-based tokenization এটি মূলত শব্দ-ভিত্তিক (word-based) এবং অক্ষর-ভিত্তিক (character-based) পদ্ধতির একটি সমন্বিত রূপ বা "মিডল গ্রাউন্ড", যা আগের দুই পদ্ধতির সীমাবদ্ধতাগুলো দূর করে। নিচে এর কাজ করার প্রক্রিয়া ও সুবিধাগুলো বিস্তারিত তুলে ধরা হলো:

Let's </w>	do</w>	token	ization</w>	!</w>
------------	--------	-------	-------------	-------

- মূল নীতি: এই পদ্ধতির মূল মন্ত্র হলো— বেশি ব্যবহৃত শব্দগুলোকে ভাঙা হবে না, কিন্তু বিরল বা জটিল শব্দগুলোকে ছোট ছোট অর্থপূর্ণ অংশে (**subwords**) ভাগ করা হবে। যেমন: "dog" একটি অতি পরিচিত শব্দ, তাই একে 'd', 'o', 'g' এভাবে আলাদা না করে একটি একক আইডি দেওয়া হয়। কিন্তু "dogs" শব্দটির ক্ষেত্রে মডেল বুঝতে পারে যে এর মূলে রয়েছে "dog" এবং সাথে একটি 's' যুক্ত হয়ে এর অর্থ সামান্য পরিবর্তন করেছে।
- অর্থের ধারাবাহিকতা বজায় রাখা: সাবওয়ার্ড টোকেনাইজেশনের মাধ্যমে মডেল একই মূল থেকে আসা বিভিন্ন শব্দের মধ্যে সম্পর্ক বুঝতে পারে। যেমন: **token**, **tokens**, **tokenizing**, এবং **tokenization**—এই শব্দগুলো যে একে অপরের সাথে যুক্ত এবং এদের অর্থ কাছাকাছি, তা মডেল সহজেই ধরতে পারে। এছাড়া একই ধরনের সাফিক্স বা প্রত্যয় (যেমন: modernization, immunization) যুক্ত শব্দগুলোর গঠনগত মিলও মডেল বুঝতে পারে।
- টোকেন চেনার উপায়: এই টোকেনাইজারগুলো সাধারণত বিশেষ চিহ্ন ব্যবহার করে বোঝায় যে কোনটি একটি শব্দের শুরু আর কোনটি শব্দের বাকি অংশ। উদাহরণস্বরূপ, BERT টোকেনাইজার প্রিফিক্স ব্যবহার করে।
 - **token**: শব্দের শুরু।
 - **ization**: এটি শব্দের একটি অংশ যা আগের অংশের সাথে যুক্ত হবে।

- সুবিধাসমূহ:

- এটি ভোকাবুলারি সাইজ কমিয়ে আনে, কারণ অনেক বড় শব্দকে ছোট ছোট সাধারণ সাবওয়ার্ড দিয়ে প্রকাশ করা যায়।
- এটি তথ্য শেয়ার করতে সাহায্য করে, অর্থাৎ একটি শব্দের প্রিফিক্স বা সাফিক্স থেকে অন্য শব্দের অর্থ বুঝতে পারে।
- এটি "**Out-of-vocabulary**" (OOV) বা অজানা শব্দের সমস্যা অনেকলাঘব করে এবং একই সাথে সিকোয়েন্সের দৈর্ঘ্যও খুব বেশি বড় হতে দেয় না।

বর্তমানে ইংরেজি ভাষার প্রায় সব **State-of-the-art** (সেরা মানের) মডেলগুলো এই সাবওয়ার্ড টোকেনাইজেশন পদ্ধতি ব্যবহার করে